INTERNSHIP TECHNICAL TASK

Full Stack · Next.js · NestJS · MongoDB

AI Tools Encouraged · Ownership Required

Deadline	7 days from the date you receive this document
Time estimate	10 – 20 hours (AI tools are encouraged and expected)
Deliverable	One GitHub repository link + ARCHITECTURE.md inside it
We assess	Understanding, architecture thinking, honest AI use
Submit via	Email to hiring contact — subject line format in Section 7

1. WHAT WE ARE LOOKING FOR

This is an internship. We do not expect years of experience. We expect curiosity, honesty, and the ability to own what you produce. Every submission is reviewed against four questions:

Can you use AI tools well?

AI assistants are part of the modern dev workflow. We want to see you use them to move faster and smarter — not as a substitute for understanding.

Do you understand what you built?

You must be able to explain every part of your submission in plain language. If AI wrote something, you still need to understand it.

How do you think about architecture?

Even at intern level, structure matters. Can you explain why things are organised the way they are?

Do you take ownership?

Bugs and gaps are fine. Not knowing your own code, or missing the brief entirely, is not.

2. THE TASK — BUILD A TASK MANAGER APP

Build a simple full-stack Task Management application. Keep the scope small and focused. A clean, working app with clear structure beats a large one with broken features.

Backend — NestJS + MongoDB

- User registration and login with JWT authentication
- Full CRUD API for tasks (Create, Read, Update, Delete)
- Task fields: title, description, status, due date, priority
- Filter and sort tasks by status and priority
- Route protection using NestJS Guards
- Input validation using class-validator
- Mongoose schemas with correct field types and structure

Frontend — Next.js

- Login and Register pages
- Task dashboard — list, create, edit, and delete tasks
- Filter tasks by status and priority
- Pages protected by auth state — redirect to login if not authenticated
- Clean and readable UI — functional matters more than beautiful
- Environment variables via .env.local — no hardcoded secrets

3. BONUS CHALLENGE — OPTIONAL, BUT IT STANDS OUT

These are not required. Completing either one will significantly strengthen your application. Partial is fine — a well-explained partial attempt scores higher than a broken full attempt.

Bonus A — Real-Time Notifications using WebSockets

When any task changes, all connected users see the update instantly — no page refresh.

- Use NestJS @WebSocketGateway with Socket.io or ws
- Emit events on task create, update, and delete from the NestJS service
- Subscribe on the Next.js side and update the task list reactively
- Only authenticated users may connect — reject unauthenticated sockets
- Show a visible toast or badge when a real-time event arrives

In ARCHITECTURE.md: explain the WebSocket lifecycle, why you chose your approach, and how you handle reconnections.

Bonus B — Role-Based Access Control (RBAC)

Extend the auth system with two roles: Admin and Member. Permissions must be enforced at the API level — UI-only restrictions do not count.

- Admin: view all tasks, edit any task, delete any task, assign tasks to others
- Member: view and manage their own tasks only
- Implement using a custom @Roles() decorator and RolesGuard in NestJS
- Store role on the User schema: 'admin' | 'member'
- Return 403 Forbidden (not 404) on unauthorized actions
- Seed the DB with one Admin and two Member users for testing

In ARCHITECTURE.md: explain how role is embedded in the JWT, how RolesGuard works, and why you chose this approach.

4. ARCHITECTURE DOCUMENT — MANDATORY

Include ARCHITECTURE.md in the root of your repository. This file is not optional.

This is the primary document we use to judge how well you understand your own work. It must cover:

#	Section	What to write
1	**System Overview**	A short paragraph on what the app does and how the three layers connect.
2	**Folder Structure**	Explain how both projects are organised and why you structured them that way.
3	**Database Schema**	Paste your Mongoose schemas and explain each field — its type and purpose.
4	**API Endpoints**	List every endpoint: method, path, auth required, request body, response shape.
5	**Auth Flow**	Explain JWT end-to-end: register → login → token → protected route.
6	**AI Tools Used**	Which tools, for which parts, and what you reviewed or changed after AI generated it.
7	**Decisions & Trade-offs**	What choices did you make and why? What would you improve with more time?

5. WHAT TO SUBMIT

One public GitHub repository containing the following:

Required	Optional but impressive
• README.md — how to run the project locally, step by step • ARCHITECTURE.md — your architecture document (Section 4) • /backend — your NestJS application • /frontend — your Next.js application • .env.example — sample environment variables, no real secrets • AI_LOG.md — which AI tools you used and for what	• Docker Compose to run both services with one command • Live deployment on Vercel, Railway, or Render • Unit tests for at least one NestJS service • Completion of Bonus A or Bonus B from Section 3

6. AI TOOLS POLICY

⚠ Read This Before You Start

We are an AI-friendly team. Using GitHub Copilot, ChatGPT, Cursor, or any other tool is encouraged. The rules are simple:

- You must be able to explain every part of your submission in plain language
- You must document AI use honestly in AI_LOG.md
- Saying "the AI wrote this and I reviewed it" is completely fine — we reward honesty

- Submitting AI output you have not reviewed or cannot explain will disqualify you

In your follow-up interview you will walk through your code live. **Candidates who are transparent about AI use and genuinely understand their work always score higher than those who look polished but cannot explain what they submitted.**

7. HOW TO SUBMIT

When your repository is ready, send an email to the hiring contact:

- **Subject line: [Intern Task] Your Full Name — Task Submission**
- Body: your GitHub repository link
- Body: 3–5 sentences on what you built and what you are most proud of
- Deadline: 7 days from the date you received this document

The repository must be public or reviewer access must be granted before the deadline.

Good luck — we are rooting for you.

If anything is unclear, reach out before you start. We would rather answer a quick question than review something built on the wrong assumption.